

P1782R0: Local contract restrictions

Audience: EWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

June 17, 2019

Introduction

Discussion of the mechanisms for controlling contract evaluation and assumption continues; see my [P1494R0](#) as well as [P1490R0](#) for a review of some recent concerns involving optimization. Papers like [P1332R0](#) have also proposed significant extensions to those mechanisms for the purposes of better control and scalability.

Fundamentally, a contract (that is, a *contract-attribute-specifier*) is a boolean expression whose evaluation and significance can be controlled non-locally (by, *e.g.*, the continuation mode). The common protocol (across many files of diverse authorship) for such control is the principal reason that that contracts exist as a language feature. The alternative, per-library approach is of course already available:

```
namespace my_lib {
    struct Stuff { /*...*/ };

    bool check_expensive(), keep_going();
    bool bad_news(const Stuff&);

    void do_stuff(const Stuff &s) {
        if(check_expensive() && bad_news(s))
            if(keep_going()) std::fprintf(/*...*/);
        else throw /*...*/;
        s.do_it();
    }
}
```

It is also worth noting that concerns about optimization and assumptions in particular are subject to the normal implementation-supplied control over such analyses (`-fno-assume-contracts` wouldn't surprise anyone and would be implied by `-O0`). All that being said, the usability concerns raised for C++20 and for the future are very real.

This paper proposes, as a bug fix to address these issues, to replace the current *contract-level* with a set of *restrictions* that limit the effect of the global controls, to be applied when local conditions (including the recency of a contract's introduction) make it unsafe to take full advantage of the consistent control. The local restrictions only reduce the set of behaviors available via the global controls for a construct; the only direct control (along the lines of [P1334R0](#)) provided is a guaranteed check. The proposal also stops well short of an extensible roles system as proposed (though not for C++20) by [P1332R0](#). One additional global control is also proposed.

Proposal

Global

Augment the build level and violation continuation mode with one additional global control, the *assumption mode*, that when *off* restricts contract condition evaluation to those that are guaranteed to be evaluated. That is, it suppresses the controversial passage from `[dcl.attr.contract.check]/4`:

- it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to `false`, the behavior is undefined.

Local

In the place where one of `default`, `audit`, or `axiom` may currently appear, allow any combination of the following modifiers with the given semantics:

<code>tentative</code>	the expression may not be assumed if it is not evaluated
<code>halt</code>	the continuation mode is taken to be <i>off</i> for this contract
<code>static</code>	the expression is never evaluated
<code>audit</code>	the expression may not be evaluated if the build level is not <i>audit</i>
<code>always</code>	the expression is always evaluated at all build levels

A new contract may be introduced with the `tentative` restriction to avoid adding undefined behavior to existing interfaces; this is similar to the `continue` proposed in [P1290R1](#). Note that the contract condition may still be assumed in code that *follows* it if it is checked and the continuation mode is *off*.

If the global assumption mode and continuation mode are *on*, a `halt` contract may still be assumed in code that follows it. The `static` restriction replaces the use of `axiom` for static analysis, removing the overloading with its use for assumptions for optimization. The role played by `audit` is much the same as in the current draft. The principal purpose of `always` is to allow the contract syntax to be used with no global control at all (in which case the only reasonable behavior is checking).

Use cases

P1332R0 proposes several features to satisfy its extensive set of use cases. Several of these are covered by this simpler proposal:

1. The `ignore` semantic as a choice for a contract level is provided by the global assumption mode.
2. The use of the `%review` tag is addressed by the `tentative` restriction, perhaps used in conjunction with a throwing handler.
3. The manual choice of semantics is supported by the `always` modifier (in the case where it is not trivially equivalent to certain `if` statements).
4. The mixed treatment of `audit` and normal contracts is supported by generally using a continuation mode of *on* and adding `halt` to checks past which it is known to be unsafe to continue.

Most of the others may be addressed elsewhere:

1. The modes per-library may be much easier to realize as per-module (since per-translation-unit doesn't mix well with `#include`).
2. The `check_maybe_continue` optimization barrier is realized more generally by `std::observable` from P1494R0.

`check_always_continue` is not addressed, but it appears to be outside the scope of the language to restrict optimization in such a fashion.

Wording

Wording to be provided (post-haste) in case of EWG approval.